

МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ
УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ
“БРЕСТСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ
УНИВЕРСИТЕТ”
ФАКУЛЬТЕТ ЭЛЕКТРОННО-ИНФОРМАЦИОННЫХ СИСТЕМ

Кафедра интеллектуальных информационных технологий

Отчет по лабораторной работе №3

Специальность ПО-13

Выполнил:
Босак В.Ю.
Студент
группы ПО-13
Проверил:
А.Д.Кулик
13.03.2026

Брест 2026

Цель работы: приобрести навыки применения паттернов проектирования при решении практических задач с использованием языка Python.

Задание 1.

3) Проект «Бургер-закусочная». Реализовать возможность формирования заказа из определенных позиций (тип бургера (веганский, куриный и т.д.)), напиток (холодный – пепси, кока-кола и т.д.; горячий – кофе, чай и т.д.), тип упаковки – с собой, на месте.

Должна формироваться итоговая стоимость заказа.

Выполнение: Код программы:

```
class Tour:
    def __init__(self):
        self.transport = None
        self.hotel = None
        self.food = None
        self.excursions = []
        self.price = 0

    def __str__(self):
        return f"Тип:
{self.transport}, {self.hotel},
питание: {self.food}, экскурсии:
{self.excursions}, цена:
{self.price}"

class TourBuilder:
    def __init__(self):
        self.tour = Tour()

    def add_transport(self,
transport, cost):
        self.tour.transport =
transport
        self.tour.price += cost
        return self

    def add_hotel(self, hotel,
cost):
        self.tour.hotel = hotel
        self.tour.price += cost
        return self

    def add_food(self, food, cost):
        self.tour.food = food
        self.tour.price += cost
        return self

    def add_excursion(self,
excursion, cost):
```

```
        self.tour.excursions.append(excursion)
        self.tour.price += cost
        return self
```

```
    def build(self):
        return self.tour
```

```
print("=== Туристическое бюро ===")
builder = TourBuilder()
```

```
print("\nВыберите транспорт:")
print("1 - Самолет (300$)")
print("2 - Поезд (150$)")
print("3 - Автобус (100$)")
choice = input("Ваш выбор: ")
if choice == "1":
    builder.add_transport("Самолет", 300)
elif choice == "2":
    builder.add_transport("Поезд", 150)
else:
    builder.add_transport("Автобус", 100)
```

```
print("\nВыберите проживание:")
print("1 - Отель 5* (500$)")
print("2 - Отель 3* (300$)")
print("3 - Хостел (100$)")
choice = input("Ваш выбор: ")
if choice == "1":
    builder.add_hotel("Отель 5*", 500)
elif choice == "2":
    builder.add_hotel("Отель 3*", 300)
else:
    builder.add_hotel("Хостел", 100)
```

```
print("\nВыберите питание:")
print("1 - Все включено (200$)")
print("2 - Завтрак+ужин (100$)")
print("3 - Без питания (0$)")
choice = input("Ваш выбор: ")
if choice == "1":
    builder.add_food("Все включено", 200)
elif choice == "2":
    builder.add_food("Завтрак+ужин", 100)
else:
```

```

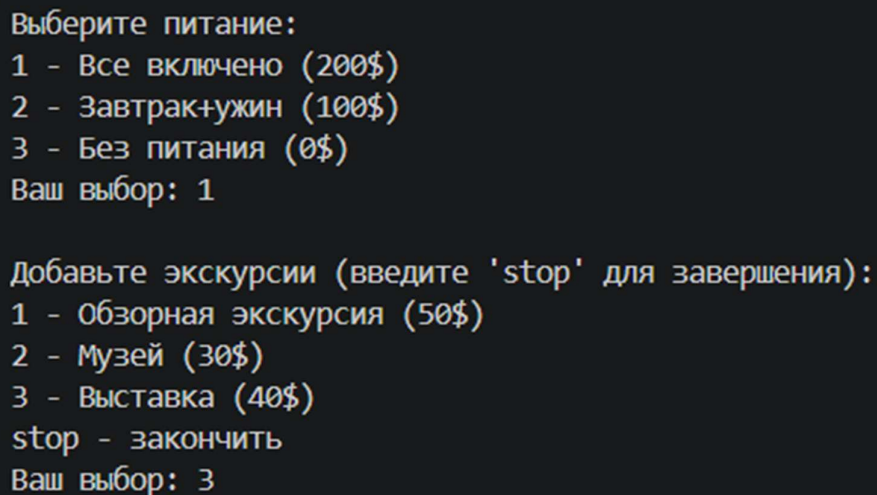
        builder.add_food("Без питания",
0)

print("\nДобавьте экскурсии
(введите 'stop' для завершения):")
while True:
    print("1 - Обзорная экскурсия
(50$)")
    print("2 - Музей (30$)")
    print("3 - Выставка (40$)")
    print("stop - закончить")
    choice = input("Ваш выбор: ")
    if choice == "1":
        builder.add_excursion("Обз
орная экскурсия", 50)
    elif choice == "2":
        builder.add_excursion("Муз
ей", 30)
    elif choice == "3":
        builder.add_excursion("Выс
тавка", 40)
    elif choice == "stop":
        break

tour = builder.build()
print("\n=== Ваш тур ===")
print(tour)

```

Рисунки с результатами работы программы



```

Выберите питание:
1 - Все включено (200$)
2 - Завтрак+ужин (100$)
3 - Без питания (0$)
Ваш выбор: 1

Добавьте экскурсии (введите 'stop' для завершения):
1 - Обзорная экскурсия (50$)
2 - Музей (30$)
3 - Выставка (40$)
stop - закончить
Ваш выбор: 3

```

Задание 2. Третья группа заданий (поведенческий паттерн)

3) Проект «ИТ-компания». В проекте должен быть реализован класс «Сотрудник» с субординацией (т.е. должна быть возможность определения кому подчиняется сотрудник и кто находится в его подчинении). Для каждого сотрудника помимо сведений о субординации хранятся другие данные (ФИО,

отдел, должность, зарплата). Предусмотреть возможность удаления и добавления сотрудника.

Код программы: from datetime import datetime

```
class File:
```

```
    def __init__(self, name, size, ext):
```

```
        self.name = name
```

```
        self.size = size
```

```
        self.ext = ext
```

```
        self.created = datetime.now()
```

```
    def __str__(self):
```

```
        return f"Файл: {self.name}.{self.ext}, размер: {self.size}KB, создан: {self.created.date()}"
```

```
class Directory:
```

```
    def __init__(self, name):
```

```
        self.name = name
```

```
        self.children = []
```

```
    def add(self, component):
```

```
        self.children.append(component)
```

```
    def get_size(self):
```

```
        return sum(c.size if isinstance(c, File) else c.get_size() for c in self.children)
```

```
    def __str__(self):
```

```
        return f"Папка: {self.name}, размер: {self.get_size()}KB"
```

```
def show_structure(item, level=0):
```

```
    indent = " " * level
```

```
    print(indent + str(item))
```

```
    if isinstance(item, Directory):
```

```
        for child in item.children:
```

```
            show_structure(child, level + 1)
```

```
print("=== Файловая система ===")
```

```
root = Directory(input("Имя корневой папки: "))
```

```
while True:
```

```
    print("\n1 - Добавить папку")
```

```
    print("2 - Добавить файл")
```

```
    print("3 - Показать структуру")
```

```
    print("4 - Выход")
```

```
    choice = input("Выбор: ")
```

```

if choice == "1":
    name = input("Имя папки: ")
    root.add(Directory(name))
    print(f'Папка '{name}' добавлена")

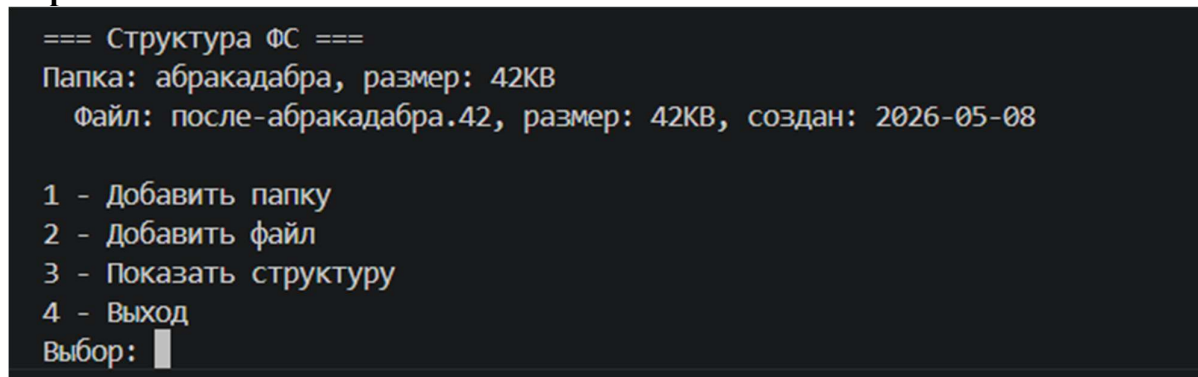
elif choice == "2":
    name = input("Имя файла: ")
    size = int(input("Размер (KB): "))
    ext = input("Расширение: ")
    root.add(File(name, size, ext))
    print(f'Файл '{name}.{ext}' добавлен")

elif choice == "3":
    print("\n=== Структура ФС ===")
    show_structure(root)

elif choice == "4":
    break

```

Скриншот:



```

=== Структура ФС ===
Папка: абракадабра, размер: 42KB
  Файл: после-абракадабра.42, размер: 42KB, создан: 2026-05-08

1 - Добавить папку
2 - Добавить файл
3 - Показать структуру
4 - Выход
Выбор: █

```

Задание 3. Вторая группа заданий (структурный паттерн)

3) Проект «Расчет зарплаты». Для задания, указанного во втором пункте («ИТкомпания») реализовать расчет зарплаты с выводом полного отчета. Порядок вывода сотрудников в отчете – по старшинству для каждого отдела.

Выполнение:

Код программы:

```

import random
from datetime
import datetime

class File:
    def
    __init__(self,

```

```

name, size,
ext):

self.name = name

self.size = size
    self.ext
= ext

self.created =
datetime.now()

    def
__str__(self):
    return
f"Файл:
{self.name}.{self
f.ext}, размер:
{self.size}KB,
создан:
{self.created.da
te()}"

class Directory:
    def
__init__(self,
name):

self.name = name

self.children =
[]

    def
add(self,
component):

self.children.ap
pend(component)

    def
get_size(self):
    return
sum(c.size if
isinstance(c,
File) else
c.get_size() for

```

```

c in
self.children)

    def
__str__(self):
    return
f"Папка:
{self.name},
размер:
{self.get_size()
}KB"

def
collect_all(comp
onent, result):

result.append(co
mponent)
    if
isinstance(compo
nent,
Directory):
        for
child in
component.childr
en:

collect_all(chil
d, result)
    return
result

print("===
Создание
файловой системы
===")
root =
Directory(input(
"Имя корневой
папки: "))

while True:
    print("\n1 -
Добавить папку")
    print("2 -
Добавить файл")

```



```
        print("3 -  
Закончить  
создание ФС")  
        choice =  
input("Выбор: ")
```

```
        if choice ==  
"1":  
            name =  
input("Имя  
папки: ")
```

```
root.add(Directo  
ry(name))
```

```
        elif choice  
== "2":  
            name =  
input("Имя  
файла: ")  
            size =  
int(input("Разме  
р (KB): "))  
            ext =  
input("Расширени  
е: ")
```

```
root.add(File(na  
me, size, ext))
```

```
        elif choice  
== "3":  
            break
```

```
all_items =  
collect_all(root  
, [])
```

```
print("\n===  
Обычный порядок  
вывода ===")  
for item in  
all_items:  
    print(item)
```

```
print("\n===  
Случайный
```

```
порядок вывода
===")
random.shuffle(a
ll_items)
for item in
all_items:
    print(item)
```

Рисунки с результатами работы программы

```
=== Обычный порядок вывода ===
Папка: ау, размер: 12КВ
Файл: ап.2, размер: 12КВ, создан: 2026-05-08
Папка: кв, размер: 0КВ

=== Случайный порядок вывода ===
Папка: кв, размер: 0КВ
Папка: ау, размер: 12КВ
Файл: ап.2, размер: 12КВ, создан: 2026-05-08
```

Вывод: приобрел навыки применения паттернов проектирования при решении практических задач с использованием языка Python.